

VERSION 0.1

UGTOMS Local Substrate Coding Agent Guide

Source-grounded architecture, measured 32K laptop deployment, and bounded local use

A practical and evidence-bounded guide to the UGTOMS/NHDF substrate, its clean-room implementation, and a complete Qwen3-30B-A3B local coding runtime on an RTX 5070 Ti Laptop GPU.

Source	UGTOMS_Local_Substrate_Coding_Agent_Guide_v0.1.md
Source SHA-256	b98438b44ac45d2650fb805df1bf45ca8aba139fbce1ea4a7e5a2ecb15969ffe
Source format	markdown

Executive result

The complete Qwen/Qwen3-30B-A3B-Instruct-2507 model is locally executable on the tested NVIDIA GeForce RTX 5070 Ti Laptop GPU with 12,227 MiB of reported VRAM. The stored model has 30,532,122,624 total parameters, while approximately 3.3B mixture-of-experts parameters are active per token. This distinction is important: the whole model must be stored, but each generated token does not perform dense 30.532B-parameter computation.

The deployable profile is a substrate-governed external-codec hybrid. The low-bit weight encoding is GGUF/IQ2_M from the ggml/Bartowski ecosystem. UGTOMS and NHDF provide the typed contract, provenance, resource policy, validation state, evidence chain, and fail-closed launch boundary. They do not claim to have invented IQ2_M or to have made dense weights disappear mathematically.

The fresh 32K gate passed 4/4 functional prompts, offloaded 49/49 reported layers, allocated a 32,768-token q4 K/V cache, and peaked at 11,068 MiB on the 12,227 MiB GPU. The measured margin was 1,159 MiB. A separate 64-token llama-bench run measured 442.151809 prompt tokens/s and 132.502673 generated tokens/s. These are useful and responsive local rates, but they are not a broad coding-accuracy result and they are not filled-32K throughput.

Outcome classification. The full-model runtime, 32K allocation profile, and one bounded general-purpose coding-agent workflow are validated on the named machine. The coding result is a narrow acceptance gate, not broad parity with a hosted frontier coding agent.

The substrate and agent outcomes must be read separately:

Evidence track	Current disposition
Direct deterministic substrate replay, without a model or agent	PASSED
Bounded generic local coding-agent gate	PASSED
Focused one-defect SCLP repair live gate	PASSED, fully disclosed repair
Historical broad four-file substrate-authoring live gate	FAILED by timeout
External-codec 32K model artifact	VALIDATED
Legacy NHDF-native scalar codec	QUALITY_FAILED

This guide remains report version 0.1, but it documents the breaking v0.2 substrate document schemas. Report version, semantic kernel/application version, and machine-readable schema discriminator are separate version axes.

Evidence classes used in this guide

Every important statement belongs to one of these classes.

Class	Meaning	Authority
Source fact	Recorded in a named source or registry	Source and recorded hash
Clean-room implementation	Reimplemented locally from typed semantics	Code plus tests
Measured result	Produced by a named local evidence run	Evidence JSON and hardware scope
Limitation	A boundary, failure, or missing validation	Must remain visible
Future hypothesis	A proposed direction not yet promoted	No present authority

A successful PDF render would only package these claims. It would not upgrade a source statement, passing unit test, or hypothesis into a measured result.

Substrate identity and provenance

Source facts: authority is stratified

The substrate is a finite, typed, deterministic geometric-topological state-and-operator algebra. It is not merely a renderer, not merely a query-first calculus, and not a synonym for model quantization. The support/compatibility/guard chain is the event-admission discipline inside the substrate rather than the identity of the substrate as a whole.

The authority strata are recorded in `substrate/kernel/contract.json` and summarized below.

The semantic kernel identity remains `ugtoms-kernel-v0.1`. Its current schema is `ugtoms-kernel-contract-0.2`; the registry, profile, and application schemas are `ugtoms-profile-registry-0.2`, `ugtoms-profile-0.2`, and `ugtoms-application-manifest-0.2`. The reference evidence schema is `ugtoms-sclp-reference-evidence-0.2`. A v0.1 semantic ID or filename therefore does not denote the superseded 0.1 schema.

Stratum	Role	Repository record
Historical motif	Provenance only; unsupported claims excluded	Kernel source record
NHDF v0.1	Normalized base closure	<code>sources/NHDF_Formal_Specification_v0.1.pdf</code>
UGTS-KC 2.0	Early typed executable algebra	External hash record, not redistributed
UGTS-KC 3.6	Content-addressed referential layer	External hash record, not redistributed
SCLP 3.6.2	Foundational corrective profile	<code>substrate/profiles/scpl-foundational.json</code>
UGTS-GN 1.1	Event-admission and execution discipline	External hash record, not redistributed
NHDF v0.3 CCD	Optional later CCD profile	<code>substrate/profiles/nhdf-v0.3-ccd.json</code>

NHDF v0.1 supplies the normalized closed composition: log-polar cells, cell-local nondegenerate zero sets, typed parity and routing, causal motion, cone/circle/sphere/apex relations, projection, and an explicit update into the next generation. Early UGTS supplies the wider typed algebra for geometry, topology, patterns, fields, kinematics, dynamics, events, transition, and lineage. UGTS 3.6 supplies content-addressed definitions, instances by definition reference, and explicit pipelines.

NHDF v0.3 CCD is preserved as a selectable profile. CCD collision and time-of-impact blocks are not silently made part of every substrate program, and they are not relabelled as language-model weight operators.

Clean-room implementation boundary

Historical and external packages remain read-only. Where redistribution rights were unclear, the repository retained source hashes and reimplemented the typed equations and invariants without copying the application code. The compact implementation is divided as follows.

- `substrate/kernel/contract.json` records kernel identity, state groups, operator mappings, symbol firewall, execution chain, source strata, and contamination policy.
- `substrate/profiles/registry.json` selects profiles without allowing a later profile to replace the base silently.
- `substrate/applications/ugtoms-sclp-reference-v0.1.json` binds the first-party direct replay to exact kernel, profile, and evidence hashes. Its ten selected-profile claim-coverage rows fail closed when coverage is missing, unknown, duplicated, assigned to the wrong profile, false, or unresolved.
- `substrate/evidence/application_proofs.json` separately records older bounded application evidence without importing those payloads into the kernel.
- `substrate/extensions/registry.json` starts generated extension proposals in QUARANTINED state and requires human review.
- `src/nhdf_edge/substrate_contract.py` validates kernel/profile bindings, source records, application manifests, and extension proposals.
- `src/nhdf_edge/substrate_graph.py` implements Merkle-bound typed definitions,

definition-hash-bound instances, ordered step-hash-bound pipelines, deterministic transitive hashes, topological resolution, cycle rejection, and endpoint-hash/typed-port-bound next-generation feedback records.

- `src/nhdf_edge/substrate_runtime.py` implements the small executable

primitives: log-polar addressing, parity, topology, cone/circle/sphere fields, SCLP keys, guarded events, bounded routing, kinematics, replay, and lineage.

- `src/nhdf_edge/substrate_packing.py` cleanly reimplements fixed-width pose

and motion words, optional shared binary16 log-polar tables, canonical sparse component records, bounded content-addressed recipes, SplitMix64 random access, and render-only generated display instances.

- `substrate/AGENT_CONTRACT.md` is the digest-verified substrate instruction

installed into the isolated local coding-agent environment.

Entertainment systems, wallets, UI layers, GIS providers, and learned teacher stacks do not become kernel authority because they use the same vocabulary.

The SCLP 3.6.2 corrective profile

SCLP 3.6.2 is a first-class foundational correction, not discarded legacy and not the whole kernel. Its repository profile is `substrate/profiles/sc1p-foundational.json`; executable reference mechanisms are in `src/nhdf_edge/substrate_runtime.py`.

The profile keeps these claims separate:

- an ordinary cone implicit field;
- an exact finite-cone signed-distance function;
- a certified translational sweep interval;
- a sphere signed-distance support and paired-sphere support relation;
- a spatial log-polar metric, Jacobian, velocity, and acceleration;
- deterministic one-bit jitter whose interval stays strictly inside the event

guard margin;

- a source half-turn bundle map and a distinct reflective Klein quotient;
- comparison BST ordering, bounded L-system geometry, and radix-prefix

refinement as different operators;

- 20/18/14/12-bit quantized state in separate contiguous and Morton 64-bit

layouts.

That list is the selectable profile's semantic envelope, not a statement that one application executed every mechanism. The direct reference replay executes the finite-cone SDF, one sphere SDF, an actual translational opposite-sign sweep bracket, spatial log-polar metric and kinematics, the one-bit jitter certificate, reflective Klein gluing, bounded BST-T/L-system-style routing, and both packed-key layouts. It explicitly bypasses paired-sphere support, circle/distributed-apex geometry, the source half-turn map, and radix-prefix refinement. A bypass is visible non-coverage, not inferred success.

SCLP results still hand off to support, compatibility, guard, certification, transition, and lineage. Packed width is not by itself semantic compression; omitted state needs a reconstruction or error contract.

Symbol firewall

The symbol firewall prevents convenient notation from collapsing distinct state or claim classes.

Protected concepts	Canonical notation	Required separation
Cone length, time, tick	T_cone, t, X	Geometry, causal time, modular address
Golden ratio, phase	phi_g, phase_phi	Constant versus evolving angle
Two log radii	rho_jitter, rho_spatial	Residual magnitude versus physical chart
Two epsilons	epsilon_jitter, epsilon_guard	Jitter bound must remain below guard
Four bit roles	payload_parity_bit, topology_parity_bit, jitter_control_bit, branch_control_bit	No shared hidden state
Two tree forms	comparison BST, radix trie	Ordering versus prefix refinement
Two topology maps	half-turn, reflective Klein	Source twist versus quotient gluing
Three cone claims	implicit, finite SDF, sweep	Relation, exact distance, certified bound

Circle, sphere, and apex are also typed separately. A circle may be a cone base, cross-section, or projection. A sphere is an SDF support. An apex is a local or distributed anchor. None of these names implies the others.

Referential closure without an unrestricted fixed point

Source fact

Within one generation, definitions form a content-addressed acyclic typed graph. A pipeline declares its order. Coordinates are not identity; definition identity, generative address, stable instance identity, and lineage remain separate.

Clean-room implementation

src/nhdf_edge/substrate_graph.py implements DefinitionNode, DefinitionInstance, Pipeline, FeedbackEdge, and SubstrateGraph. Definition content hashes cover typed domain, codomain, dependency content hashes, evaluation phase, parameters, equation, units, bounds, failures, ports, and provenance. Instances bind their referenced definition hash. Pipelines bind their ordered step hashes, generation, domain, codomain, and typed adjacency. Feedback binds source and target content hashes, named typed ports, and exact generation boundaries. Same-generation dependencies are topologically sorted; missing references, phase inversions, hash mismatches, port/type mismatches, generation violations, and cycles fail visibly.

A feedback record is deliberately outside that same-generation DAG. It may connect an observable or residual at generation n only to an input at generation $n+1$:

```
definition DAG at generation n
-> observable or residual at n
-> explicit provenance-bearing feedback edge
-> input at generation n+1
```

`FeedbackEdge.fixed_point_claim` is false, and `SubstrateGraph.fixed_point_engine` is false. This is source-grounded referential closure. It is not an implementation or proof of an unrestricted self-modifying fixed-point engine, and it makes no general convergence, stability, existence, or uniqueness claim.

The composed referential flow is:

```
input/residual
-> log-polar address and metric
-> cell-local nondegenerate zero set
-> payload_parity_bit, topology_parity_bit, jitter_control_bit, branch_control_bit
-> bounded BST, L-system, or radix routing
-> causal vector kinematics
-> cone, sphere, SDF, sweep relation, and projection
-> support, compatibility, guard, and certificate
-> atomic transition
-> lineage and novelty
-> explicit feedback into generation n+1
```

Optional stages require an explicit bypass. They cannot be reordered or fused without a declared equivalence scope and evidence.

Spatial work remains a future boundary

Source-grounded retained principles

`substrate/incubator/spatial-evidence-ledger.json` records a quarantined future profile. It retains stable identity distinct from coordinates, typed observations and relation surfaces, uncertainty intervals, atomic patches with pre-state and post-state hashes, checkpoints, replay, repeat-scan deltas, lineage, and novelty.

Future hypothesis

A later nontraditional spatial knowledge graph may combine three views:

1. a content-addressed definition graph;
2. an instance, relation, event, and lineage graph;
3. an optional spatial index and projection graph.

This is not promoted today. HGT, TGN, teacher models, H3, GeoNames, OpenStreetMap, application games, Android code, wallets, and UI adapters remain outside the kernel. Learned components may rank or propose; they cannot become geometric, identity, or provenance authority without deterministic review.

Direct and legacy application evidence

These records demonstrate useful compact generation or packing. They do not define the kernel and their payloads are not loaded into the local model.

First-party deterministic substrate replay

examples/ugtoms_sclp_reference.py composes the clean-room graph, runtime, and packing primitives directly. It is not a model-generated result. The current sealed records are:

Record	Bytes	SHA-256
Kernel contract	10,187	9b5fa7cfff4483129e80e1c234055d57e0f79a574dcd2e131c418bbe0259448c3
Profile registry	1,003	aa1d788808ebd624dcff435538cd4e63f13004bf84101956ffe4119f14b14152
Selected nhdf-v0.1 profile	3,429	f70ceec98d9029f057e7ac71187a18fe28e9108ee8e79b45130d0942a22a8625
Selected sclp-foundational profile	5,258	4c21f81bf73c863065963b9296f053781079c19c6c2b22ac5783d7ea957e9d13
Registered, unselected nhdf-v0.3-ccd profile	3,514	56c728d74cb03642b46ff3d3854d540590b01ddd903e747b23a2d2f2d8a36c62
Reference example	31,380	9c9c2c09d5832984bbe6c4faace9eb9fc139c6ff2c4649f718d4296b61e7120
Application manifest	13,111	42451914715eb1f8be85481457068f310db644c034ae1fceedf479d8314065c
Deterministic replay evidence	10,154	1d0dc094d9649b667739fc2a202316097ae980baf5585cba22b8e33675282a42

Two executions produced byte-for-byte identical output equal to the committed evidence. The application validator reports one evidence record, 12 mappings across all nine kernel mapping categories, and exact coverage of ten requirements: four declared by NHDF v0.1 and six by SCLP. The graph contains ten Merkle-bound definitions. Its feedback edge is typed, binds both endpoint hashes and named ports, and crosses exactly from generation zero to one. may_propose_extensions and may_promote_extensions are false.

The actual 32-iteration cone sweep retains an opposite-sign interval [0.702539065154, 0.702539065387]; it makes no earliest-impact claim. The replay also records both packed-key round trips, a safe one-bit jitter interval, bounded routing and resource traces, atomic lineage, and an exact stable render-only display prefix. Its explicit bypasses and non-claims are part of the evidence, not omissions to be silently promoted.

KC3D Grove scene: explicit packing

The KC3D392 Grove scene is a 21,798-byte deterministic binary scene with 66 explicit nodes and shared-resource references. It demonstrates compact scene packing. It does not demonstrate recipe-generated populations.

Separate Grove Ring recipe: generated display population

The separate bounded recipe regenerated 1,024 render-only display instances. The prototype-plus-recipe runtime asset was 4,984 bytes; the comparison using 1,024 real ECS movers was 162,274 bytes. The reported ratio was 32.56x and the reported reduction was 96.93%, with exact prior-prefix stability.

Those generated copies are display instances only. They do not independently acquire collider, graph, saved-object, or gameplay identity. The 4,984-byte recipe result must not be attributed to the 21,798-byte explicit KC3D scene.

Clean-room local packing reproduction

src/nhdf_edge/substrate_packing.py reproduces the bounded mechanism with a new wire format rather than importing the legacy code. In its 1,024-display fixture, the shared LUT plus one prototype component pack and one recipe pack occupied 1,999 bytes. Materializing 1,024 real packed component records used 26,279 bytes. That is 13.15x smaller, a 92.39% reduction. A comparison with 1,024 binary32 4x4 transform matrices (65,536 bytes) is 32.78x smaller, a 96.95% reduction.

The prefix from 64 to 1,024 derived instances was byte-for-byte stable because each lineage lane is random-access from the recipe identity and ordinal. The same critical boundary applies: the derived records are render-only. This measurement is not evidence for compressing model weights, exogenous meshes, textures, observations, or independently mutable game entities, and it makes no GPU speed claim.

KSGP/SGP4 reconstruction proof

The 5,793-byte KSGP1 seed/timeline artifact reconstructed 328 states with zero mismatch flags. The exact recorded maximum deltas were:

Quantity	Maximum delta
Position	2.656269518315808e-7 km
Velocity	7.841169468777989e-10 km/s

This demonstrates bounded seed-based reconstruction and lineage for that application. It does not place generic SGP4 mathematics inside the substrate kernel.

Local model and external-codec boundary

Source and implementation facts

The validated local-agent artifact is packs/qwen3-30b-a3b-iq2m-32k-q4kv. Its manifest is NHDF_HYBRID_MANIFEST.json, and its fresh evidence is evidence/functional_gate.json.

Property	Exact value
Total model parameters	30,532,122,624
Approximate active parameters/token	3.3B
BF16 tensor bytes	61,064,245,248
BF16 weight memory	58,235.4 MiB
Referenced IQ2_M payload	9,870,270,464 bytes
Physical GPU capacity	12,227 MiB

BF16 weights alone are approximately 4.76 times the reported GPU capacity, before K/V cache and compute buffers. The referenced IQ2_M file is 6.1867 times smaller than the BF16 tensor bytes, an 83.84% byte reduction. These ratios are file/storage comparisons, not measures of semantic equivalence or answer quality.

The hybrid manifest references the verified GGUF payload in place rather than copying another 9.87 GB into the Git repository. GGUF/IQ2_M remains the weight codec. The 32K profile additionally

uses q4_0 for the K and V caches; that K/V choice is not the weight codec.

The runtime is pinned llama.cpp build 10720 at revision f8dbcd61893702976f9ab03be89c2b9f436d532c. The substrate seals the payload, runtime, contract, and evidence records; declares bounded GPU/context policy; and refuses ordinary launch unless the artifact has measured VALIDATED status.

Honest native-codec status

The earlier NHDF-native scalar pack is not the functional codec. Its complete 9,152,386,624-byte tensor pack loaded, executed on CUDA, and passed 531 manifest/CRC/parity checks, but its output collapsed to repeated newlines or 10000000. It is correctly classified QUALITY_FAILED. Integrity and fit did not imply useful language-model output.

Fresh 32K measurements

The tracked public snapshot of the fresh gate is metrics/local/ugtoms_local_agent_32k/functional_gate.json; the corresponding artifact record is packs/qwen3-30b-a3b-iq2m-32k-q4kv/evidence/functional_gate.json. The snapshot is 32,876 bytes with SHA-256 d56140c0a4bc97fb9fab5d3930222a494e681744570363d0f810a8e872aa01c1. It combines a four-prompt functional suite, a separate allocated-32K exact response, complete layer-offload checks, resource monitoring, and 64-token prompt/decode microbenchmarks.

Fresh gate item	Result
Functional prompts	4/4 passed
Reported layer offload	49/49
Allocated context	32,768 tokens
32K K/V cache	864.00 MiB q4_0
Peak device memory	11,068 MiB
Device headroom	1,159 MiB
64-token prompt rate	442.151809 tok/s
64-token decode rate	132.502673 tok/s

The three prompt samples were 299.733, 445.363, and 581.359 tok/s, standard deviation 140.840844. The three decode samples were 106.623, 147.103, and 143.782 tok/s, standard deviation 22.473620.

The four functional outputs included exact OK, exact 323, a coherent integrity-versus-quality explanation, and a correct small Python function. The separate 32K allocation run also returned exact OK. All 49 reported layers were offloaded.

The 32K result validates allocation, residency, a short execution, and the declared resource margin. It does not validate answer quality after filling all 32,768 positions. The profile is therefore called **32K validated** only with that precise scope.

A 48K configuration is treated as **fragile**, not validated. It is not the launcher default and was not promoted into the artifact contract. This guide does not invent a stable 48K claim from an experimental ability to allocate or start. Any future 48K profile must independently satisfy the

reserve, filled context, output quality, and repeatability gates.

What the token rates mean in practice

Short-context microbenchmark

At 132.502673 decode tokens/s, raw decode takes about 7.547 ms per token. In idealized arithmetic, 100 generated tokens take about 0.755 seconds and 300 take about 2.264 seconds. At 442.151809 prompt tokens/s, the 64-token benchmark prompt takes about 0.145 seconds to process.

For ordinary local coding chat, that short-context decode rate is comfortably interactive. Actual response time also includes model startup, request handling, prompt length, tool execution, file reads, test runs, and UI overhead. A benchmark rate is not the same as end-to-end task completion.

Separate prior filled-context measurement

A prior, separately scoped filled-context run measured 22,440 prompt tokens, 2,297.477 prompt tokens/s, and 25.189 decode tokens/s.

Derived practical quantity	Approximate time
Prefill 22,440 tokens	9.77 s
One decoded token	39.70 ms
Decode 100 tokens	3.97 s
Decode 300 tokens	11.91 s

Long batched prompt processing can report a higher tokens/s number than the short 64-token prompt test because GPU prefill and fixed overhead have a very different shape. Decode slows at a long active context because each new token must attend over more retained context and interact with a larger K/V working set.

The 25.189 tok/s result is still usable for reading and editing assistance, but it is visibly slower than the short-context 132.502673 tok/s result. This prior measurement must remain separate from the sealed fresh gate: it does not prove filled-32K quality, does not validate 48K, and should be promoted only when its raw evidence is registered alongside the artifact.

Local coding-agent components

The local interface uses a pinned project-local OpenCode client and a loopback-only llama.cpp server. No OpenAI or other hosted API account is used for inference.

Component	Role
START_LOCAL_CODER.cmd	Double-click trusted-Python desktop entry point
scripts/start_local_coder_gui.ps1	Scrubbed Windows interpreter discovery and launch
scripts/local_coder_gui.py	Readiness, download, resident-server, session, and prompt UI
scripts/setup_local_coder.ps1	Installs pinned OpenCode 1.18.25 locally

Component	Role
scripts/start_local_coder.ps1	PowerShell setup-and-launch wrapper
scripts/local_coder.py	Validates target, config, contract, artifact, and runtime
configs/opencode_nhdf_local.json	Local-only provider and permission contract
scripts/benchmark_local_coder_agent.py	Bounded native-tool, context, and repair gate
src/nhdf_edge/server.py	Owns loopback model-server lifecycle

scripts/local_coder.py requires a Git working tree, verifies the pinned configuration and installed substrate contract, validates the exact 32K/q4 artifact profile, starts the owned server, and stops it when the client exits. The isolated configuration disables cloud providers, plugins, MCP, external skills, sharing, and automatic updates. Review mode leaves ordinary edits and shell commands at OpenCode's approval boundary. Named web, external-path, destructive, commit, and push spellings are denied by the application policy.

Those controls are useful scope reduction, not complete command mediation or an OS sandbox. Work mode adds OpenCode --auto, so an operation that falls through the finite deny rules can be approved automatically. Use Work only on a committed or otherwise recoverable Git worktree, inspect the diff, and do not treat it as equivalent to a hardened VM or container.

Measured bounded generic coding-agent acceptance

The final recorded gate is committed as

metrics/local/ugtoms_local_agent_32k/coding_agent_gate.json (original run directory run-20260831T210715.054259Z), 10,074 bytes, with SHA-256

253fe50d62fe70ea6ca82b6a481639197e0a98988e2a8b530339ebbf518c7e6b. It reported the served allocation from meta.n_ctx as exactly 32,768 tokens; the model's larger training-context metadata was deliberately not accepted as served capacity.

Agent-gate item	Measured result
Native typed tool-call probe	passed
Exact synthetic retrieval prompt	21,997 tokens
Retrieved needle	exact match
Disposable repair wall time	30.299467 s
Recorded Edit calls	exactly 1
Recorded Bash calls	exactly 1
Independent fixture tests	4/4 passed
Files changed	only src/intervals.py
Git HEAD	unchanged

The model inspected the local fixture, diagnosed a touching-interval condition, changed < to <=, ran exactly python -m pytest -q, and stopped after the passing result. The revised gate permits at most one identical pre-edit pytest and one post-edit pytest because that is a normal coding workflow; every Bash call must still be the exact offline command and only one Edit is allowed. A preceding fresh run made the correct repair and passed 4/4, but was rejected by the old one-Bash cardinality rule after performing both checks. It was not relabelled as a pass. This observed

planning variability is why the capability claim remains bounded.

Substrate-aware live-agent status

The generic pass above must not be relabelled as substrate awareness, and the direct deterministic replay must not be relabelled as an agent result.

The focused live gate asked the local agent to make one minimal repair to a declared SCLP feedback defect and then satisfy deterministic replay and Git-scope checks. Its 10,268-byte evidence file is

metrics/local/ugtoms_local_agent_32k/substrate_focused_repair/evidence.json, SHA-256 3a0669b40de948330a451670dd61a7baf2e03771f2d8bb128354b1da830ba4b3. It records status: PASSED and all_gates_passed: true.

Focused repair check	Measured result
Agent wall time	34.241295 s
Recorded tool mutations/commands	Exactly one edit, exactly one bash
Independent fixture tests	3/3 passed
Deterministic replay	Two byte-identical independent runs
Changed path	Only src/sclp_repair.py
Git HEAD	Unchanged

The fixture and prompt fully disclosed the defect, same-generation feedback n to n , and the exact required correction, bounded feedback n to $n+1$. The result proves instruction-following and tool compliance for that one specified repair. It does not prove independent diagnosis, broad substrate understanding, general coding competence, compression, or preventive sandboxing.

The earlier broad four-file SCLP authoring attempt is already a measured negative result, not pending. On 31 August 2026 it passed server, configuration, 32K context, and native-tool-call prechecks; verified the expected 4/4 failing unimplemented clean-room baseline; then timed out after 1,200 seconds while producing repeated oversized malformed Edit calls. Its normalized evidence file is 2,750 bytes, SHA-256

c0bab4824086ea9497dae5d82d519994cffced593d44b183d4fe392e797a69e8, and records status: FAILED and all_gates_passed: false at metrics/local/ugtoms_local_agent_32k/substrate_full_authoring_failure/evidence.json.

This failure does not negate the direct replay or generic-agent passes. It only refuses the broader autonomous-authoring claim that the timed-out run was meant to test.

Setup and daily use

Prerequisites

- Windows with a compatible NVIDIA driver;
- the exact pinned CUDA 12.8 runtime DLLs in the validated Program Files CUDA location, with the recorded hashes;
- Microsoft Visual C++ 2015-2022 x64 Redistributable;

- a trusted registered Python 3.10 or newer with tkinter;
- enough free GPU memory to satisfy the artifact preflight.

The large model file is intentionally not committed to the free Git repository. The GUI can download it from the pinned immutable source revision. The verified OpenCode archive is committed, so the normal release flow does not require Node.js, npm, an API key, or a hosted inference account.

End-to-end desktop use

1. Double-click `START_LOCAL_CODER.cmd` in the repository root.
2. If needed, select **Install Client**. The GUI expands the committed, hash-pinned OpenCode 1.18.25 archive into `.local-coder`.
3. If needed, select **Download Model**. The transfer can resume; final promotion occurs only after the exact size and SHA-256 match.
4. Select the Git repository you want the agent to inspect or edit.
5. Select **Start Resident Model** once. The 32K model remains resident across prompts.
6. Use Review mode by default. Select Work only after reading and accepting its per-session warning. **New Session** clears conversation state without unloading the model; **Stop** releases it.

The five cards expose Client, Model, Runtime + CUDA, GPU, and Artifact state. All five were probed READY on the target machine. Diagnostics are compacted and may be truncated for display; they are not a byte-for-byte raw log. Full GUI operation and corrupt-model recovery are documented in `docs/LOCAL_CODER_GUI.md`.

Command-line setup

From the repository root:

```
python -m pip install -e ".[dev, runtime]"
powershell -ExecutionPolicy Bypass -File .\scripts\setup_local_coder.ps1
nhd-f-edge verify .\packs\qwen3-30b-a3b-iq2m-32k-q4kv
```

The setup script prefers the committed verified client archive. Model inference after installation is local and does not consume an API budget. Hardware, electricity, and initial download/storage costs still exist.

First trusted interactive launch

Use a full payload verification on the first launch after the model changes or moves:

```
python .\scripts\local_coder.py "C:\path\to\your\git-worktree"
```

Routine launch

--quick is a compatibility spelling for existing scripts. It does not skip or reduce verification; the same strict 9.87 GB SHA-256 remains mandatory after the payload is locked against concurrent replacement:

```
python .\scripts\local_coder.py --quick "C:\path\to\your\git-worktree"
```

Equivalent wrapper invocation:

```
.\scripts\start_local_coder.ps1 -Arguments @(
    "--quick",
    "C:\path\to\your\git-worktree"
)
```

One noninteractive request

Place `--run` last because all remaining arguments are passed to OpenCode:

```
python .\scripts\local_coder.py `
    --quick `
    "C:\path\to\your\git-worktree" `
    --run "Inspect the repository, explain the failing test, and do not edit yet."
```

The normal launcher asks for approval before edits and ordinary shell commands. Its finite policy denies named classes of network, external-directory, destructive, commit, push, and delegation operations. This narrows scope but does not prove that every equivalent spelling is prevented.

Validation rubric

Substrate implementation checks

```
python -m pytest `
    tests\test_substrate_contract.py `
    tests\test_substrate_graph.py `
    tests\test_substrate_runtime.py `
    tests\test_substrate_packing.py `
    tests\test_substrate_reference_application.py `
    tests\test_substrate_pdf.py `
    -q

nhdf-edge substrate-verify --repository .

nhdf-edge substrate-validate-app `
    .\substrate\applications\ugtoms-sclp-reference-v0.1.json `
    --repository .

python .\examples\ugtoms_sclp_reference.py --output .\reference-first.json
python .\examples\ugtoms_sclp_reference.py --output .\reference-second.json
```

These tests check typed contracts, exact selected-profile evidence coverage, Merkle and transitive content hashes, typed ports and generation boundaries, topological ordering, cycle rejection, next-generation feedback, symbol separation, geometry/kinematics behavior, compact packing, byte-identical reference replay, deterministic PDF output, pypdf text validation, and Poppler page rendering. The application validator must report 12 mappings and ten covered profile requirements. Passing these checks establishes implemented invariants and one direct replay, not a live-agent result or broad scientific or application validity.

Runtime acceptance

Claim	Minimum evidence	Refusal condition
Artifact integrity	Full payload/runtime/manifest hashes	Any mismatch
Useful output	All four functional prompts pass	Any failed acceptance rule
GPU residency	49/49 offload and monitored peak	Partial offload or monitor absent
32K profile	Cache allocated and exact short response	Allocation or response failure
Resource safety	At least 512 MiB reserve	Peak exceeds declared limit
Throughput	At least 80 decode tok/s in fresh gate	Mean below threshold

The current recorded result passes this runtime rubric with 1,159 MiB headroom and 132.502673 decode tokens/s.

Coding-agent acceptance

Start the sealed 32K server in one terminal:

```
nhdf-edge serve .\packs\qwen3-30b-a3b-iq2m-32k-q4kv `
--port 18080 `
--threads 4 `
--startup-timeout 300 `
--request-timeout 600
```

Then run the bounded agent gate in another terminal:

```
python .\scripts\benchmark_local_coder_agent.py `
--server-url http://127.0.0.1:18080 `
--opencode-exe .\local-coder\node_modules\opencode-ai\bin\opencode.exe `
--config .\configs\opencode_nhdf_local.json `
--output-root .\metrics\local\coding_agent_32k
```

A coding-agent acceptance record should pass all of these stages:

1. exact pinned OpenCode and configuration checks;
2. loopback model discovery with the 32,768-token limit;
3. one valid native tool-call JSON response with exact typed arguments;
4. retrieval of a deterministic needle from an approximately 22K-token synthetic context;
5. inspection and repair of a disposable Git/Python fixture;
6. recorded local read/search/edit/test tools within the fixture;
7. independent final python -m pytest -q success;
8. a source diff with tests and baseline protected;
9. all_gates_passed: true in the generated evidence.

The recorded tool audit is retrospective, not a preventive operating-system sandbox. The actual launcher permission contract remains an additional safety layer. A successful run proves only this bounded tool/context/repair workflow; it does not establish repository-scale coding accuracy.

Honest remaining limits

- IQ2_M is an external codec. There is no validated NHDF-native tensor-codec advantage today.
- The fresh 32K run allocated the cache and executed a short prompt. It did not fill 32K with meaningful content and score long-context answer quality.
- The separate 22,440-token result showed decode falling to 25.189 tok/s. Long context is usable but materially slower.
- 48K is fragile and unvalidated. It is not a supported default.
- The 4/4 functional suite is small. It does not establish perplexity, broad reasoning, security, or coding parity with BF16 or a hosted frontier model.
- Approximately 3.3B active parameters per token explain much of the high short-context speed. It must not be called dense-30B throughput.
- q4 K/V cache quality has not been broadly compared with q8 K/V or BF16 on a long-context benchmark.
- Other desktop GPU users can consume the 1,159 MiB margin. The launcher must continue to fail closed when the free-memory contract is not met.
- Sustained thermals, battery behavior, power limits, and many-hour stability are not established.
- The zero-copy artifact is workspace-relative and not self-contained. Moving the model or repository can invalidate its path/provenance contract.
- The default configuration omits cloud accounts, MCP, plugins, and subagents; its finite policy denies named web, external-directory, commit, and push patterns. Work mode is not a complete command sandbox, so safe use still requires a recoverable worktree and diff review.
- Application proofs such as Grove and KSGP demonstrate bounded packing or reconstruction. They do not prove arbitrary semantic compression.
- The direct substrate replay is deterministic code-level evidence, not a live language-model or coding-agent measurement. Its paired-sphere, circle/apex, half-turn, and radix mechanisms are explicit bypasses.
- The focused one-defect live repair passed, but the fixture and prompt fully disclosed the answer. It proves compliant execution, not independent diagnosis or broad substrate understanding.
- The retained broad substrate-authoring live attempt failed after its 1,200-second timeout. It must not be presented as a pending or passing gate.
- A generated report or PDF is evidence packaging, not validation.

Future hypotheses and next measurements

These items are proposals, not current capabilities.

1. Run and seal filled-context quality tests at increasing prompt lengths up to 32K, with retrieval, code understanding, decode speed, and failure cases.
2. Revisit 48K only if repeated resource measurements preserve the reserve and filled-context output remains useful.
3. Design a separately bounded substrate-diagnosis gate that does not disclose the exact repair in advance. Retain both the focused compliance pass and the broad timeout rather than overwriting either result.
4. Expand the generic local coding gate across larger repositories, multiple languages, multi-file edits, and adversarial tests while retaining an explicit manual authority boundary.
5. Continue native substrate-codec research only behind numeric distortion and full-model output gates; do not promote a codec merely because it fits.
6. Develop the quarantined spatial evidence profile as definition, relation, event, lineage, and spatial-index graphs with stable identity and replay.
7. Test whether substrate-addressed sparse state improves retrieval or tool selection. Learned ranking may propose; deterministic geometry and provenance must remain authoritative.

Repository evidence map

Area	Primary files
Kernel	substrate/kernel/contract.json
Profiles	substrate/profiles/registry.json, sclp-foundational.json
Spatial future	substrate/incubator/spatial-evidence-ledger.json
Application proofs	substrate/evidence/application_proofs.json
Agent contract	substrate/AGENT_CONTRACT.md
Contract validator	src/nhdf_edge/substrate_contract.py
Referential graph	src/nhdf_edge/substrate_graph.py
Runtime primitives	src/nhdf_edge/substrate_runtime.py
Compact packing	src/nhdf_edge/substrate_packing.py
Direct reference example	examples/ugtoms_sclp_reference.py
Direct reference manifest and evidence	substrate/applications/ugtoms-sclp-reference-v0.1.json, substrate/applications/evidence/ugtoms-sclp-reference-v0.1.json
Focused repair gate and passing evidence	scripts/benchmark_substrate_repair.py, metrics/local/ugtoms_local_agent_32k/substrate_focused_repair
Historical broad-authoring failure	metrics/local/ugtoms_local_agent_32k/substrate_full_authoring_failure
PDF pipeline	src/nhdf_edge/substrate_pdf.py, scripts/render_substrate_pdf.py
32K artifact	packs/qwen3-30b-a3b-iq2m-32k-q4kv
Local launcher	scripts/local_coder.py, scripts/start_local_coder.ps1
Local config	configs/opencode_nhdf_local.json
Agent gate	scripts/benchmark_local_coder_agent.py

Final scope statement

The literal benefit demonstrated today is accessibility: a complete 30.532B-parameter MoE model whose BF16 weights require 58,235.4 MiB is usable through a 9,870,270,464-byte external low-bit payload on a 12,227 MiB laptop GPU, under a typed UGTOMS/NHDF provenance and resource contract. The fresh 32K profile fits with measured headroom and short-context decode is fast enough for comfortable local interaction. A prior long-context run remained usable at a slower 25.189 tok/s.

The substrate contribution is a deterministic contract for state, operators, profiles, provenance, evidence, bounded execution, and next-generation closure. Its first-party replay is byte-identical with 12 mappings across nine categories and all ten selected NHDF v0.1 plus SCLP requirements covered; that is substrate execution, not model compression. The disclosed focused repair proves compliant execution, not independent diagnosis. IQ2_M supplies today's tensor compression. The spatial graph, an NHDF-native codec, filled-32K quality, stable 48K, and broad coding equivalence remain future work; broad substrate authoring remains a recorded failure.